PROJECT REPORT

STREAMLINING INCIDENT MANAGEMENT: ENFORCING DATA INTEGRITY WITH CLIENT SCRIPTS AND UI POLICIES

ServiceNow Administration Project

Submitted by: B.E Computer Science Engineering

Institution: Sri Ramanujar Engineering College

Academic Year: 2026–2027

Name: Anand V

TABLE OF CONTENTS

1. INTRODUCTION

- 1.1 Project Overview
- 1.2 Purpose

2. IDEATION PHASE

- 2.1 Problem Statement
- 2.2 Empathy Map Canvas
- 2.3 Brainstorming

3. REQUIREMENT ANALYSIS

- 3.1 Customer Journey Map
- 3.2 Solution Requirement
- 3.3 Data Flow Diagram
- 3.4 Technology Stack

4. PROJECT DESIGN

- 4.1 Problem Solution Fit
- 4.2 Proposed Solution
- 4.3 Solution Architecture

5. PROJECT PLANNING & SCHEDULING

- 5.1 Project Planning

6. FUNCTIONAL AND PERFORMANCE TESTING

- 6.1 Testing & Validation

7. RESULTS

- 7.1 Output Screenshots

8. ADVANTAGES & DISADVANTAGES

9. CONCLUSION

10. FUTURE SCOPE

11. APPENDIX

1. INTRODUCTION

1.1 Project Overview

The project **"Implement Client Script & UI Policy (Incident)"** focuses on enforcing data integrity and conditional field behaviors directly at the user interface level within ServiceNow Incident Management. By utilizing built-in UI Policies and custom client-side scripts, the system dynamically governs form interactions, mandatory field rules, and validation checks.

1.2 Purpose

The purpose of the project is to eliminate incomplete, inconsistent, or incorrect data submissions that can negatively affect reporting accuracy and SLA compliance. It demonstrates how client-side controls can automate form updates, manage field visibility or read-only states, and restrict invalid submissions.

2. IDEATION PHASE

2.1 Problem Statement

Incident records require consistent data entry for proper routing and resolution. Relying solely on user awareness often results in missing mandatory details, incorrect priority alignments, or unauthorized direct updates via list editing.

2.2 Empathy Map Canvas

- **End Users / Agents:** Require quick data entry paths, automatic field handling, and clear error notifications when mandatory fields are omitted.
- **Administrators / IT Managers:** Need reliable controls to maintain high data quality, consistent ticket categorization, and predictable SLA tracking.

2.3 Brainstorming

Proposed solutions included creating UI Policies for field-state manipulation, configuring automatic field population, setting up onSubmit scripts for save-time validations, and deploying `onChange` scripts for scripts to block unauthorized list modifications. `onCellEdit`

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	User Activity	Pain Point	System Response
Initiation	User creates/edits an Incident	Missing mandatory data fields	UI Policy & Client Script validation
Interaction	User alters the Impact field	Inconsistent priority settings	Auto-populates Urgency and sets fields read-only
Validation	User attempts to save/ submit	Bypassing required assignments	`onSubmit` script blocks save and shows error box
List Editing	User edits records via list view	Uncontrolled state alterations	`onCellEdit` script intercepts and cancels change

3.2 Solution Requirement

- **Functional Requirements:** The system must dynamically trigger UI policies on high-impact incidents, validate mandatory assignment fields prior to save, auto-set dependent fields, and block unauthorized list edits.
- **Non-Functional Requirements:** Lightweight client-side execution, seamless integration with the global Incident table, and role-compliant architecture.

3.3 Data Flow Diagram

User Action (Form/List Input) → UI Policy / Client Script Evaluation (onChange, onSubmit, onCellEdit) → Condition Check → Action Execution (Set Mandatory, Read-only, Error Prompt, or Save Block) → Platform Record Update.

3.4 Technology Stack

Technology	Purpose
ServiceNow ITSM	Incident management platform repository
UI Policies & Actions	Dynamic form-level field attribute control
Client Scripts	Advanced client-side logic (`onChange`, `onSubmit` `onCellEdit`)

4. PROJECT DESIGN

4.1 Problem Solution Fit

The combination of UI policies and targeted client scripts provides multi-layered protection against bad data entry. Field rules adapt dynamically when record parameters (such as Impact) change.

4.2 Proposed Solution

- **Task 1 & 2:** Create a UI Policy named **"High Impact Control"** on the Incident table triggered when `Impact = 1 - High`, making **Assignment group** mandatory and **Urgency** read-only.
- **Task 3:** Implement an `onChange` Client Script (**"Auto set urgency for high impact"**) to automatically update the urgency field.
- **Task 4:** Implement an `onSubmit` Client Script (**"Prevent save if assigned to missing"**) to block form submission if a high-impact incident lacks an assignee.
- **Task 5:** Implement an `onCellEdit` Client Script (**"Prevent state change via list edit"**) to prevent direct list updates to the State field.

4.3 Solution Architecture

The architecture comprises the ServiceNow User interacting with the underlying Incident records administrators.

Interface layer processing client-side script events, table while enforcing policies defined by system

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Phase	Main Activities	Deliverable
Planning & Ideation	Define data integrity challenges and target fields	Scope & Requirements definition
Design	Plan UI Policy conditions and script logic	Functional design blueprint
Implementation	Configure UI Policies, Actions, and Client Scripts	Configured ServiceNow instance records
Testing	Verify form validations, list blocking, and reversals	Test verification reports

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Testing & Validation

Test Case	Expected Result	Status
High Impact Trigger	UI policy triggers when Impact is set to High	Pass
Mandatory Save Check	`onSubmit` blocks save if Assigned To is empty	Pass
Successful Save	Record saves successfully when mandatory fields are filled	Pass
Reverse Condition	Reverting Impact restores normal field behavior	Pass
List Edit Block	`onCellEdit` blocks direct state changes in list view	Pass

7. RESULTS

7.1 Output Screenshots

Recommended insertion figures matching verification steps:

- **Figure 7.1** - UI Policy Configuration ("High Impact Control")
- **Figure 7.2** - `onChange` Client Script Setup for Urgency Auto-population
- **Figure 7.3** - `onSubmit` Validation Error Message for Missing Assignee
- **Figure 7.4** - Successful Incident Record Submission
- **Figure 7.5** - `onCellEdit` Alert Box Blocking List View State Updates

8. ADVANTAGES & DISADVANTAGES

Advantages

- Enforces strict data integrity and reduces incomplete ticket submissions.
- Automates repetitive user actions through dynamic form updates.
- Prevents data corruption caused by unmonitored list edits.
- Improves reporting precision and SLA compliance.

Disadvantages

- Client scripts must be written carefully to prevent performance latency on complex forms.
- Overlapping client-side rules require structured maintenance during form upgrades.

9. CONCLUSION

The project successfully demonstrates how ServiceNow UI Policies and Client Scripts (`onChange`, `onSubmit`, `onCellEdit`) work in tandem to secure data quality on Incident records. It effectively automates form constraints, enforces mandatory ownership for critical incidents, and establishes reliable user-interface boundaries.

10. FUTURE SCOPE

- Expand client-side validations to Problem and Change Management modules.
- Introduce advanced asynchronous scripting (`g_scratchpad`) to optimize form-load performance.
- Integrate automated logging for blocked list-edit attempts to audit user behavior.

11. APPENDIX

Source Code / Script References

1. onChange Client Script (`Auto set urgency for high impact`**):**

```
functiononChange(control, oldValue, newValue, isLoading) {
    if  (isLoading || newValue === '') {
        return;
    }
    if  (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
    }
}
```

2. onSubmit Client Script (`Prevent save if assigned to missing`**):**

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' && g_form.getValue('assigned_to') == '') {
        g_form.showErrorBox('assigned_to', 'Assigned To is mandatory for High impact
incidents.');
        return false;
    }
    return true;
}
```

3. onCellEdit Client Script (`Prevent state change via list edit`**):**

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('State cannot be changed from the list. Please open the Incident.');
    callback(false);
}
```